

《云计算与大数据处理》

实验报告

实验名称	分布式批处理与性能分析（实验 03）
实验日期	2026 年 11 月 3 日
专业	计科
班级	3
学号	2410311322
姓名	陶志杰
指导教师	刘锦行

一、实验环境与身份

环境沿用前两章的 .venv。本章新增的依赖只有 Dask 与 distributed，用于启动多进程集群。

项目	实测内容
操作系统 / Python	Windows 11 家庭版 / 3.12.9
解释器	D:\mumu\work\.venv\Scripts\python.exe
项目工作目录	D:\mumu\work（课程根目录）
代码仓库	https://github.com/add1223/online_data
dask / distributed	课程锁定版本，LocalCluster 本机多进程
duckdb	1.5.5（作为正确性参照）
依赖检查	pip check 无冲突

本章的代码改动、运行证据、报告和截图都同步保存在 https://github.com/add1223/online_data 的 submissions/lab03 目录。



图 1　环境证据（学号、实验号 03、run_id 同屏）

二、问题与数据

前一章已得到干净的 Silver 数据。本章要回答两个问题：分布式计算结果与单机 SQL 是否一致；增加进程能否带来速度提升。

输入是 data/silver/events.parquet，20,000 行合法事件，sha256 为 516b3d78aab91c1487d9a24979eaef514c5acb7309d0d82a6688a762836634dc。整个过程只读，不改写这份数据。

三、三组配置：w1p8、w2p8 与 w2p4

前两组只调整 Worker 数量，分区固定为 8；第三组改为两个 Worker、四个分区。

```
coursekit lab03 --workers 1 --partitions 8 coursekit lab03 --workers 2 --partitions 8 coursekit lab03 --workers 2 --partitions 4
```

配置	数据哈希	PID 数	任务数	启动秒	计算秒	总秒	SQL一致
w1p8	516b3d78…634dc	1	115	2.24	2.98	6.16	true
w2p8	516b3d78…634dc	2	115	2.50	5.62	9.25	true
w2p4	516b3d78…634dc	2	43	2.60	5.74	9.15	true

三组数据哈希相同，SQL 对照全部为 true。w2p8 比 w1p8 慢约 50%，w2p4 也只比 w2p8 略快。两万行数据规模过小，增加进程带来的调度与序列化开销超过了节省的计算时间。

另外，total_seconds 不能作为单次查询耗时。程序在主聚合之后还执行了一次分区诊断，因此合计时间中包含诊断开销，回执中也标注了 do not claim

linear speedup。

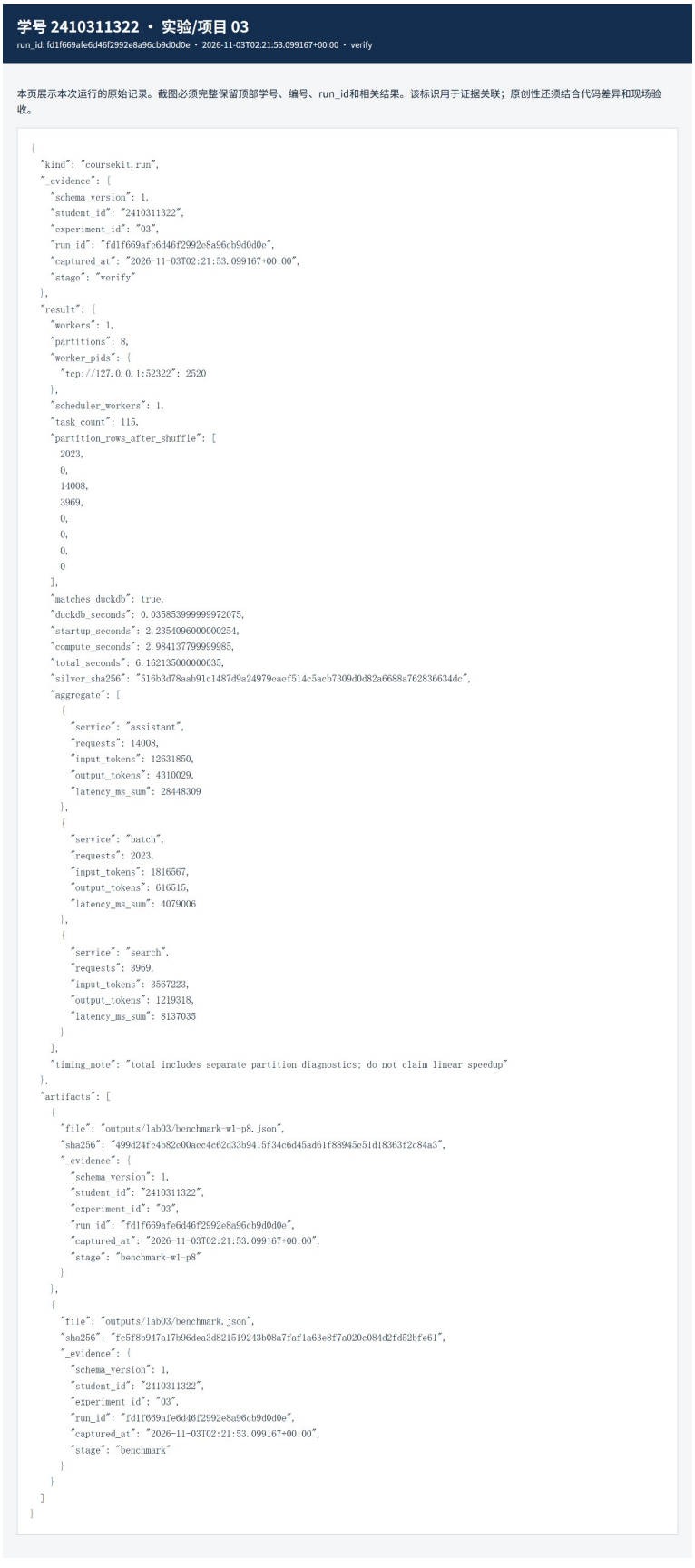


图 2　w1p8 回执（PID、分区行数、耗时）

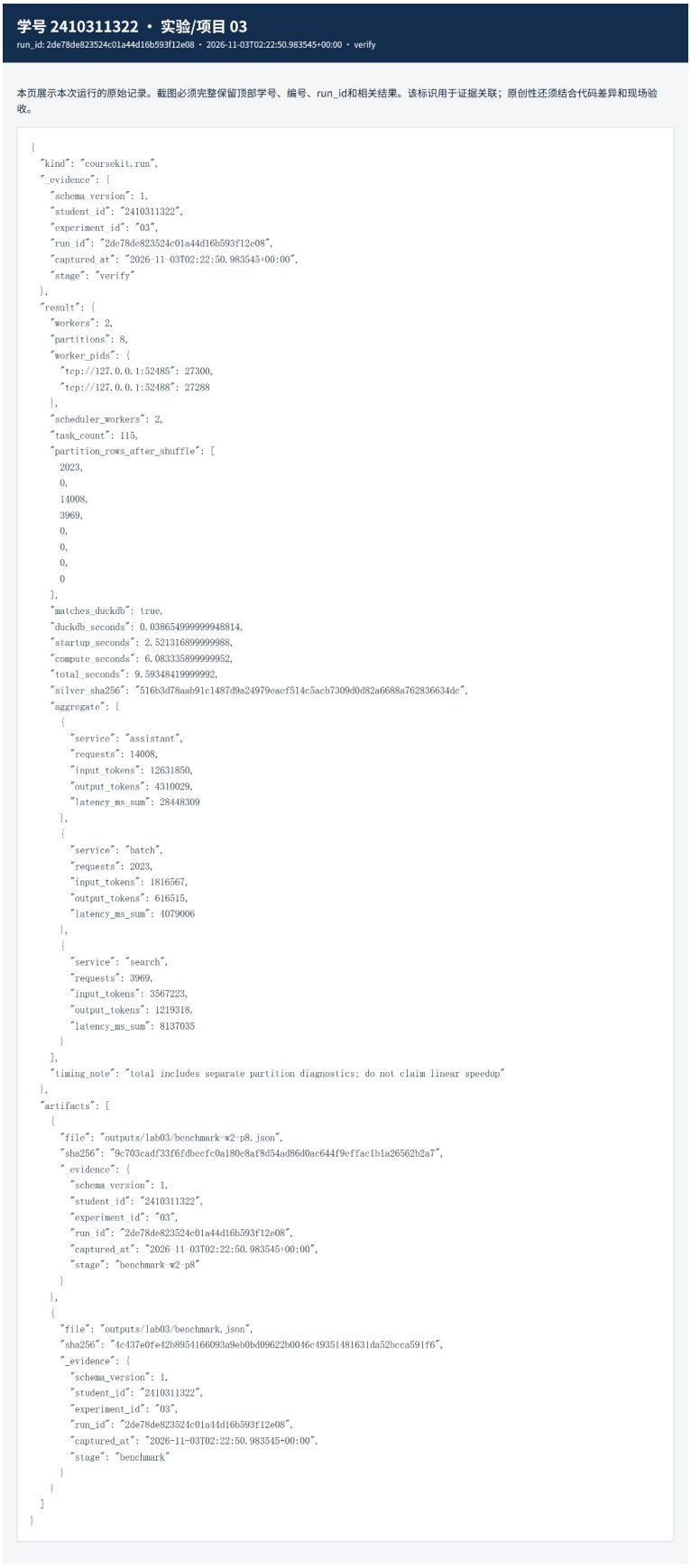


图 3　w2p8 回执（两个 Worker 的 PID 各不相同）



图 4　w2p4 回执（两个 Worker、四个分区、43 个任务）

3.1 分区与平均值手算

12 条事件，A=8、B=3、C=1。用只用于手算的哈希 A→0、B→1、C→2，按 hash % 4 分到 4 个分区，结果是 [8,3,1,0]。增加到 8 个分区仍是三个非空分区：[8,3,1,0,0,0,0,0]。因为只有三个服务键，加分区不会把热键切开。这个哈希只用于解释，真实分区编号由 Dask 计算。

平均值也不能直接取平均。分区 1 有 2 条记录、平均 10 ms；分区 2 有 8 条记录、平均 100 ms。全局平均是 (2×10+8×100)/10 = 82 ms，不是 (10+100)/2 = 55 ms。丢掉分母会把小分区的权重放大。完整手算在 submissions/lab03/hand_calculation.md。

3.2 执行结构

LocalCluster 起一个 Scheduler 和指定数量的 Worker 进程，这里 threads_per_worker=1、processes=True。Scheduler 负责任务调度，Worker 执行计算，分区是数据的计算块，线程是进程内的执行单位，四者不能混为一谈。

read_parquet 之后 repartition 到 8 或 4 个分区，再按 service 做 shuffle。Shuffle 把相同服务键的记录重新分配到同一分区，需要序列化、数据交换和临

时内存；本机进程之间同样有交换成本。任务图在 compute 之前只是依赖描述，compute 才真正执行并把小结果取回驱动进程。

四、结果正确性

每一组运行都带 matches_duckdb: true。也就是说 Dask 算出来的三行聚合，跟 DuckDB 用 SQL 算出来的逐字段一致。

service	requests	latency_ms_sum	input_tokens
assistant	14,008	28,448,309	12,631,850
batch	2,023	4,079,006	1,816,567
search	3,969	8,137,035	3,567,223

分区结果值得单独看一眼：八个分区里只有三个有数据，分别是 14008、3969、2023，另外五个是 0。因为哈希只按 service 分，而服务只有三种。所以加分区数不会把一个热键切开，写报告时不能拿 20000 ÷ 8 当观测值。

五、个人改动一：慢请求阈值 1000 → 2000

附件改一个常量，完整 diff 在 submissions/lab03/latency_aggregate.diff。

```
-SLOW_MS = 1000 +SLOW_MS = 2000
```

5.1 预测与实测

项目	预测	实测	结果
总请求数	变多	14,008 / 2,023 / 3,969，没变	未命中
延迟总和	变多	28,448,309 / 4,079,006 / 8,137,035，没变	未命中
慢请求率	增加	0.508 / 0.500 / 0.514，下降	未命中

5.2 三项全错的原因

阈值只决定「哪些请求算慢」，它不改变被统计的请求总数，也不改变耗时求和。所以前两项跟阈值无关，必然不变。

第三项的结论正好相反。门槛从 1000 提高到 2000，超过 2000 毫秒的请求更少。三个服务的平均耗时都在 2030 毫秒左右，因此慢请求率从约 75% 降到约 51%，接近「平均值以上约一半」的预期。

这次三项预测全部落空，但换来一条可复用的判断：改参数之前先分清它是筛选口径还是数据范围，口径参数只改变分类，不改变总量。

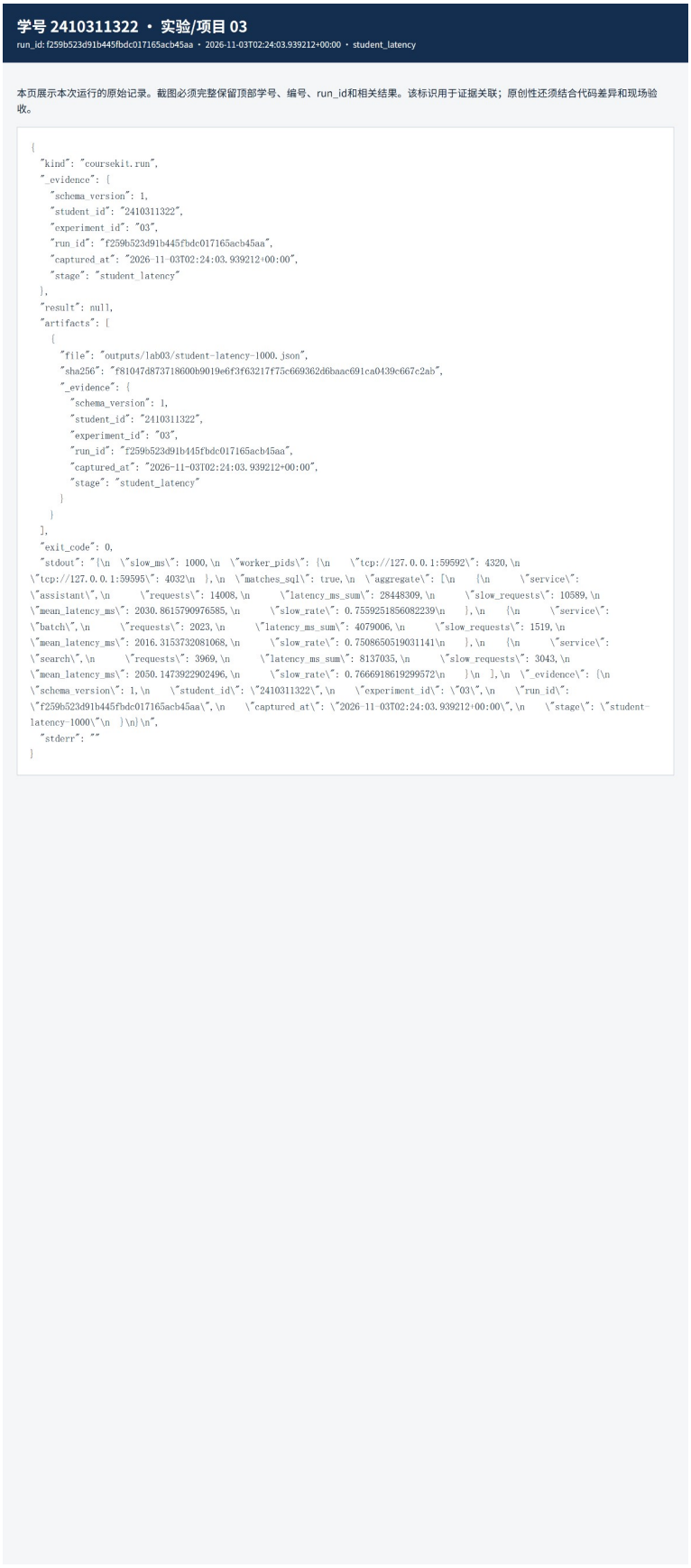


图 5　阈值 1000 的基线结果

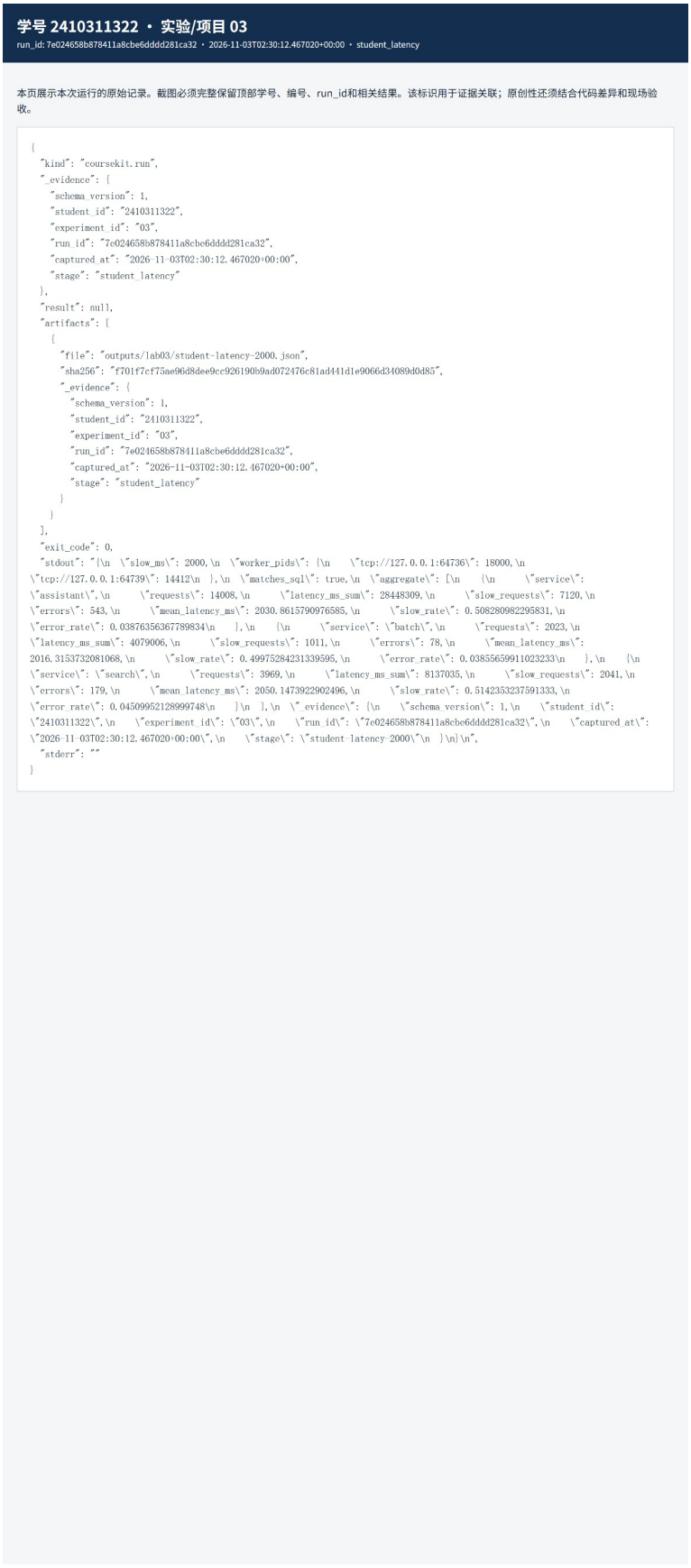


图 6　阈值 2000，并已加入 errors 与 error_rate

六、个人改动二：新增 errors 与 error_rate

这一项要在 Dask 和 SQL 两条实现里都加，不能只算一次。完整 diff 在 submissions/lab03/latency_aggregate.diff，改动落在四处：Dask 路径构造 is_error 标志并聚合；把计数列命名为 errors；DuckDB 路径增加同口径的条件求和并继续参与 assert 比较；汇总后计算 error_rate。

Dask 侧加一列标记，再进聚合：

```
df = df.assign(is_slow=(df["latency_ms"] >= SLOW_MS).astype("int64"), is_error=(df["status"] == "error").astype("int64")) actual = df.groupby("service").agg({... , "is_error": "sum"}).compute()
```

SQL 侧加一个条件求和：

```
SUM(CASE WHEN status = 'error' THEN 1 ELSE 0 END)::BIGINT AS errors
```

两条实现的字段必须同名同值，中间那行 assert 才会通过。错误率的分母是全部合法请求，不是慢请求：

```
actual["error_rate"] = actual["errors"] / actual["requests"]
```

service	requests	errors	error_rate
assistant	14,008	543	3.88%
batch	2,023	78	3.86%
search	3,969	179	4.51%
合计	20,000	800	4.00%

独立核验脚本逐行读取 Silver 复算，输出 ERROR_EXTENSION_PASS {'total_errors': 800}。800 可以手算：生成规则为每 25 条出现一条错误，20000 ÷ 25 = 800。核验器未被改动，用于交叉检查我的实现。

再从输出任选 assistant 手算三项：平均延迟 28,448,309 / 14,008 = 2030.86 ms；慢请求率 7,120 / 14,008 = 0.5083；错误率 543 / 14,008 = 0.0388。三项都与 student-latency-2000.json 一致，错误率的分母是全部合法请求。

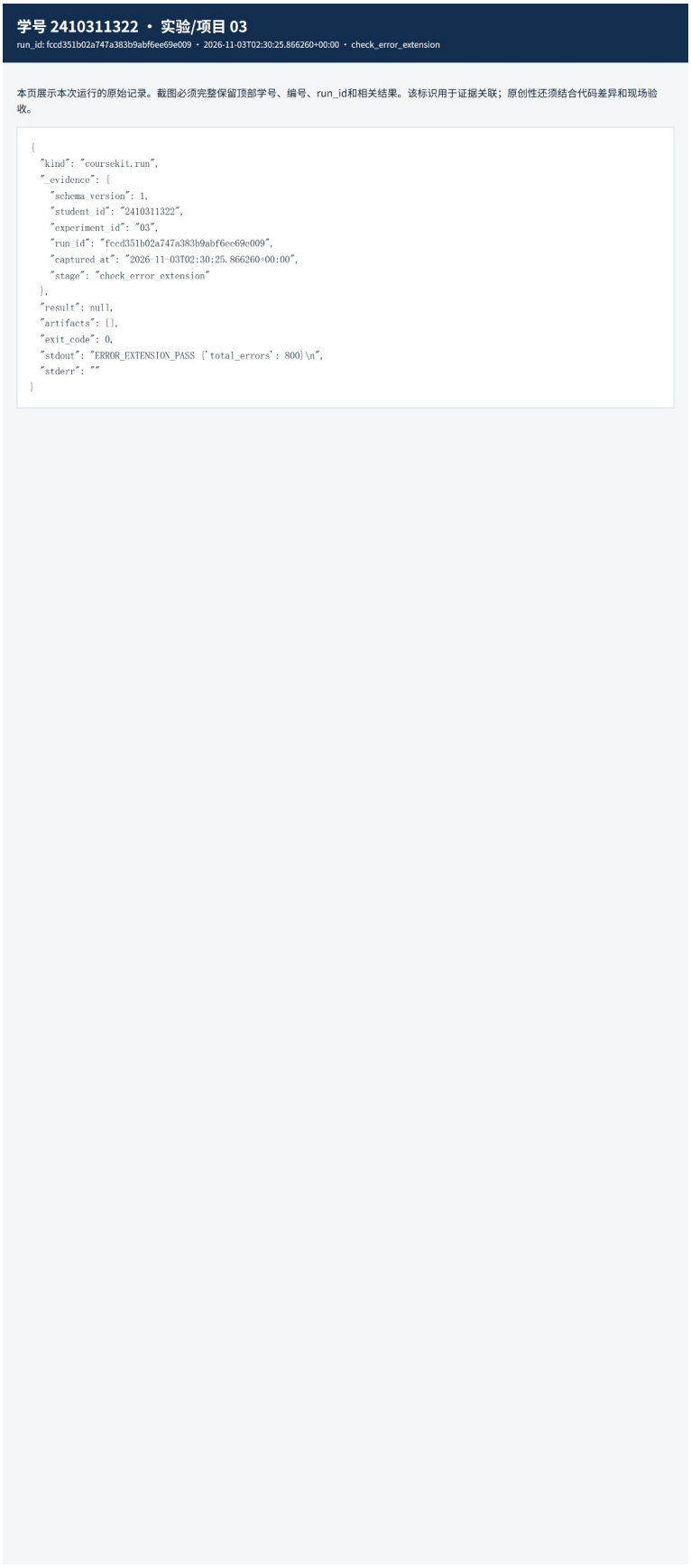


图 7　独立核验通过：total_errors = 800

七、故障与恢复

把 Worker 数量故意设为 0，观察程序是否拦截。

```
coursekit lab03 --workers 0 --partitions 8
```

结果是启动前的参数检查就把它挡住了，没有起集群：

```
ValueError: Teaching limits: 1-4 workers, 1-64 partitions. exit_code: 1
```

这属于输入校验失败，不是数据错误。恢复就是换回合法参数重跑，matches_duckdb 仍为 true，合计 9.25 秒。

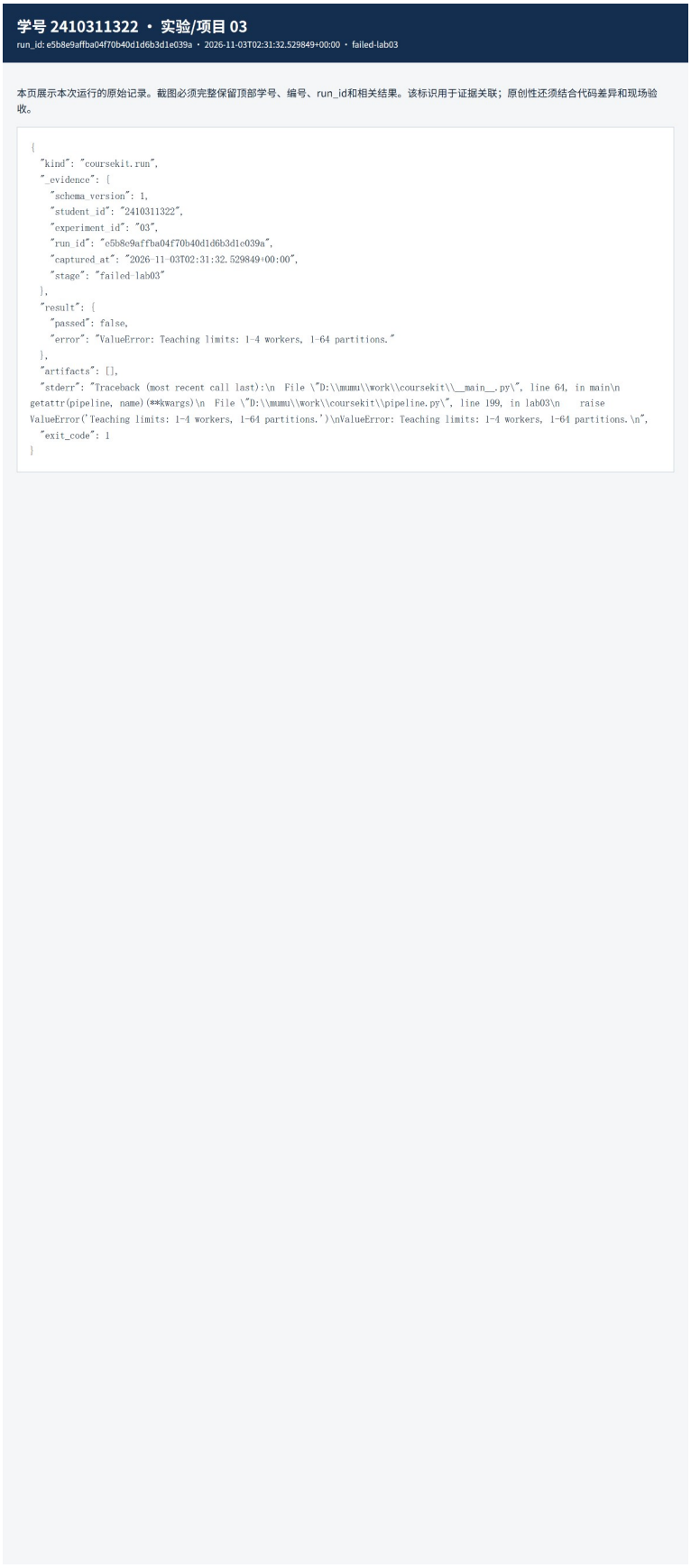


图 8　workers=0 的真实失败记录

学号 2410311322 · 实验/项目 03

run_id: 714778e5395648ffa15c2367f5b924cd · 2026-11-03T02:31:37.583655+00:00 · verify

本页展示本次运行的原始记录。截图必须完整保留顶部学号、编号、run_id和相关结果。该标识用于证据关联；原创性还须结合代码差异和现场验收。


```
{
  "kind": "coursekit.run",
  "_evidence": {
    "schema_version": 1,
    "student_id": "2410311322",
    "experiment_id": "03",
    "run_id": "714778e5395648ffa15c2367f5b924cd",
    "captured_at": "2026-11-03T02:31:37.583655+00:00",
    "stage": "verify"
  },
  "result": {
    "workers": 2,
    "partitions": 8,
    "worker_pids": {
      "tcp://127.0.0.1:64853": 4084,
      "tcp://127.0.0.1:64856": 27136
    },
    "scheduler_workers": 2,
    "task_count": 115,
    "partition_rows_after_shuffle": [
      2023,
      0,
      14008,
      3969,
      0,
      0,
      0,
      0
    ],
    "matches_duckdb": true,
    "duckdb_seconds": 0.041420700000116994,
    "startup_seconds": 2.5049468999998226,
    "compute_seconds": 5.620599900000116,
    "total_seconds": 9.250519199999871,
    "silver_sha256": "516b3d78aab91c1487d9a24979eacf514c5acb7309d0d82a6688a762836634dc",
    "aggregate": [
      {
        "service": "assistant",
        "requests": 14008,
        "input_tokens": 12631850,
        "output_tokens": 4310029,
        "latency_ms_sum": 28448309
      },
      {
        "service": "batch",
        "requests": 2023,
        "input_tokens": 1816567,
        "output_tokens": 616515,
        "latency_ms_sum": 4079006
      },
      {
        "service": "search",
        "requests": 3969,
        "input_tokens": 3567223,
        "output_tokens": 1219318,
        "latency_ms_sum": 8137035
      }
    ],
    "timing_note": "total includes separate partition diagnostics; do not claim linear speedup"
  },
  "artifacts": [
    {
      "file": "outputs/lab03/benchmark-w2-p8.json",
      "sha256": "54ba9063565cb844a9e8fec311fc9f71bc15a2337a5598c618933145aba45706",
      "_evidence": {
        "schema_version": 1,
        "student_id": "2410311322",
        "experiment_id": "03",
        "run_id": "714778e5395648ffa15c2367f5b924cd",
        "captured_at": "2026-11-03T02:31:37.583655+00:00",
        "stage": "benchmark-w2-p8"
      }
    },
    {
      "file": "outputs/lab03/benchmark.json",
      "sha256": "1b8ba9ec6b5470a5e8076d227a84413af46b32553f4f3004525ec52324b1ffb6",
      "_evidence": {
        "schema_version": 1,
        "student_id": "2410311322",
        "experiment_id": "03",
        "run_id": "714778e5395648ffa15c2367f5b924cd",
        "captured_at": "2026-11-03T02:31:37.583655+00:00",
        "stage": "benchmark"
      }
    }
  ]
}
```


图 9　恢复后重跑，结果一致

八、能力边界

项目	状态	依据
结果与单机 SQL 一致	验证	三组运行 matches_duckdb 均为 true
多进程真的启动	验证	回执中两个 Worker 的 PID 互不相同，进程数与参数一致
并行有加速	未验证	两个 Worker 反而慢约 50%，本机小数据下没有收益
跨机分布式	未验证	LocalCluster 全在本机，没有跨物理机通信
Worker 崩溃容错	未验证	没有注入进程崩溃或网络中断
生产数据量下的表现	未验证	只有 2 万行，结论不能外推

8.1 必答分析题

（1）8 个分区不等于 8 台机器或 8 个 Worker。分区是数据的计算块，Worker 是执行进程，线程又是进程内的执行单位；本机 LocalCluster 里三者不是一回事。

（2）三个服务键经 Shuffle 后很难均匀填满 8 个分区。哈希只按 service 分，最多三个非空分区；哈希碰撞还可能让非空分区更少。实测就是 3 个非空、5 个为 0。

（3）不同大小分区的平均值不能直接取平均。必须用总和除以总条数；直

接平均会把小分区的权重放大，得到 55 ms 而不是正确的 82 ms。

（4）本章 total_seconds 包含超过主计算的工作：集群启动、主聚合、分区诊断 map_partitions(len).compute()、以及退出清理。它不能当作单次查询耗时

（5）两 Worker 比一 Worker 慢时，应检查启动与调度开销、序列化与 Shuffle 成本、任务数、数据规模、缓存与磁盘状态，以及是否真的发生了并行计算。

（6）本机多个进程不足以证明生产跨机容错。它们在同一台机器、同一操作系统内，没有跨机网络分区、节点宕机、磁盘故障或副本切换，生产容错需要另做故障注入和冗余设计。

九、总结与交接

本章完成的工作：启动本机多进程集群，运行 w1p8、w2p8、w2p4 三种配置；以 DuckDB 为参照核对结果；观察分区分布；修改慢请求阈值并做对照；在 Dask 与 SQL 两侧新增错误统计；用独立脚本核验；注入一次非法参数后恢复。

主要有两点结论：一是正确性与性能是两件事，Dask 较慢不代表结果算错；

二是参数分为筛选口径与数据范围两类，改口径不会改变总量。

交接内容	路径	谁会用
Silver 明细	data/silver/events.parquet	实验 04 继续消费
个人对照结果	outputs/lab03/student-latency-2000.json	本章证据，含 errors 与 error_rate
三组性能记录	submissions/lab03/ benchmarks/*.json	实验 07 的性能讨论参照

9.1 运行记录索引

报告里每张图对应的运行记录如下。按 run_id 可以在 evidence/ 里找到原始 JSON 和同名 HTML。

图	run_id
图 1	bbef4b0fb25f49ab883930733f865511
图 2	fd1f669afe6d46f2992e8a96cb9d0d0e
图 3	2de78de823524c01a44d16b593f12e08
图 4	93515195fadb49d891b7dfea1dfa0db8
图 5	f259b523d91b445fbdc017165acb45aa
图 6	7e024658b878411a8cbe6dddd281ca32
图 7	fccd351b02a747a383b9abf6ee69e009
图 8	e5b8e9affba04f70b40d1d6b3d1e039a
图 9	714778e5395648ffa15c2367f5b924cd

十、AI 使用披露

AI 助手帮我核对了 Dask 的聚合写法、解释 total_seconds 为什么不能当查询耗时、整理报告结构。

三个预测是我在改代码前写的，三项都错，原因也是我自己对着数字想通的。错误统计的代码是我照着原有慢请求的写法扩展出来的，核验器没动。